

LUMEN

SERVIRE USER MANUAL

M0.1 Research Distribution

Continuity • Understanding • Provenance • Trust

Version 0.1 | September 2026

Illuminates.One

About this manual

This manual is the operational guide for the Lumen M0.1 Research Distribution. It is written for researchers running Lumen locally with Docker, MongoDB and an Ollama model provider. It focuses on the Servire workspace and the researcher-facing component interfaces available from it.

SCOPE M0.1 is an experimental research distribution. The screens in this manual are taken from the frozen M0.1 release and show real operating states from an end-to-end installation test.

What Servire does

Servire is the operational control plane for Lumen. It provides one workspace from which you can validate dependencies, start and stop the Stack, inspect health and logs, and open the interfaces for the managed Lumen components.

Terminology

- Lumen - the complete system, including Servire and all managed services.
- The Stack - the managed Lumen services considered together while Servire is running.
- Trace - a recorded interaction sequence captured by Vestigare.
- Replay - a Repetere experiment that re-executes a recorded trace against a selected model.
- Prepared replay - a replay made available to Fiducia for scheduled execution.

M0.1 component map

Component	Role	Default port
Fiducia	Replay orchestration and scheduling	11430
Praebere	Model provider and model lifecycle	11431
Rogare	Conversational client	11432
Pontis	Communication bridge	11435
Moderari	Orchestrator	11436
Repetere	Replay and experiment operator	11437
Vestigare	Trace recorder	11438
Servire	Operational control plane	11439
Nuntius	Bootstrap and service communication	11440

1. Before you begin

Research distribution package

Your research package should contain the Docker Compose definition, the HAProxy configuration, the Lumen Research Distribution licence and supporting documentation. Registry credentials and the Lumen installation registration key may be supplied separately.

Prerequisites

- Docker Desktop with Docker Compose available.
- Ollama installed and running on the host or another reachable machine.
- At least one suitable Ollama model already available.
- Network access to registry.illuminates.one for the initial image pull and to Illuminates.One when Lumen performs periodic authorization.

Two separate credentials

The private registry credential controls access to the Lumen Docker image. The Lumen registration key controls authorization of the installed Research Distribution. They are separate mechanisms and should be treated separately.

RESEARCH LICENCE By installing, starting, accessing or using the Lumen M0.1 Research Distribution you accept the Research Distribution licence terms presented at first start.

Persistent data

Docker Compose creates persistent volumes for the Lumen workspace, Servire data and MongoDB data. These volumes belong to the local installation and are not part of the distributed Lumen image.

2. Install and start Lumen

Authenticate to the private registry

- 1 Open a terminal in the directory containing the distribution package.
- 2 Run: `docker login registry.illuminates.one`
- 3 Enter the researcher-specific registry username and password when prompted. A successful login confirms that Docker can authenticate to the private registry.

Start the distribution

- 1 Ensure Ollama is running.
- 2 Run: `docker compose up -d`
- 3 Allow Docker to pull the frozen Lumen M0.1 image and the required supporting images.
- 4 When the containers are running, open Servire at `http://localhost:11439`. If accessing Lumen from another machine on the LAN, use the Docker host address with port 11439.

REQUIRED FILE Keep `haproxy.cfg` in the same distribution directory as the Compose file. The HAProxy service bind-mounts this file; if it is missing, the user-facing Lumen ports will not start correctly.

First start

On first start, Servire presents the Lumen M0.1 Research Licence. Read the licence and accept it to continue. Servire then provides the registration and authorization flow for the installation. A correctly authorized installation displays **AUTHORISED** together with its lease information.

3. Servire: validate and start the Stack

The Servire home page is the operational starting point. The top navigation opens each component inside the unified workspace. Stack Health summarizes the current state; Stack Actions exposes the permitted lifecycle actions; the Operational Log records what Servire and the managed components are doing.

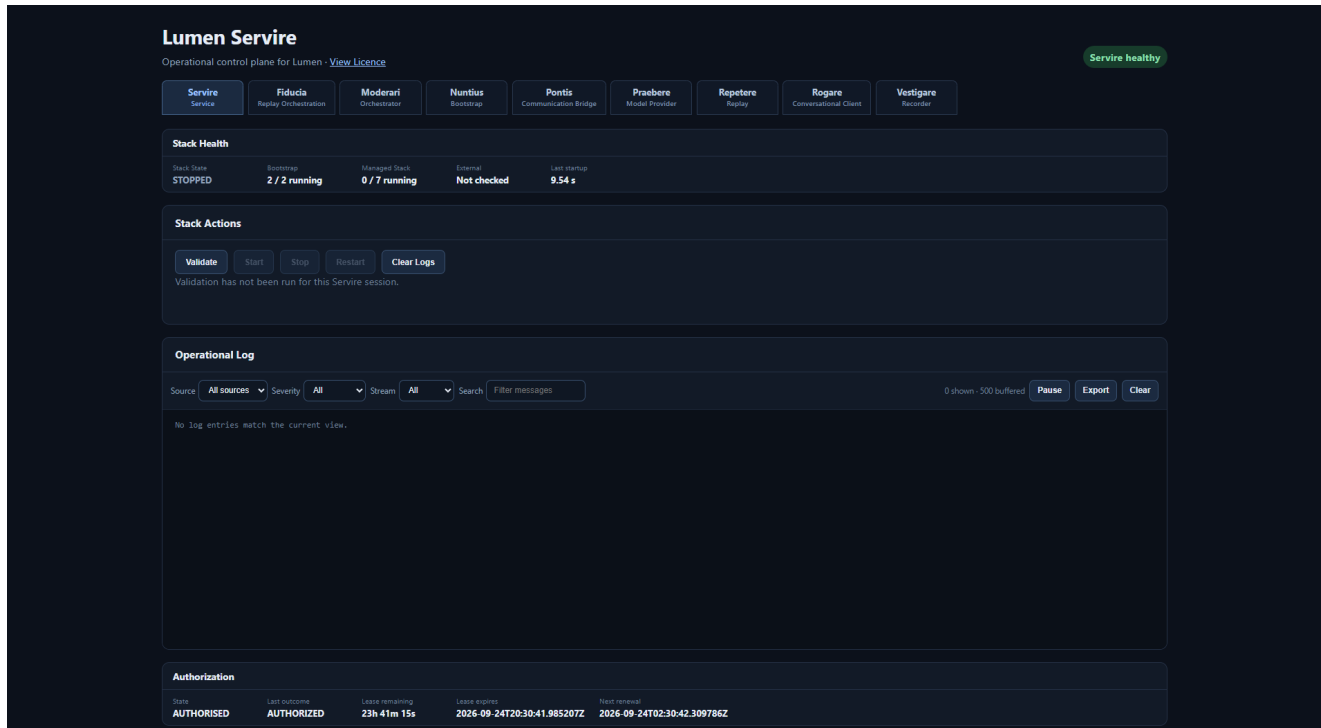


Figure 1. Servire before validation: Stack state, lifecycle controls, operational log and authorization status.

Validate

Select Validate before starting the Stack. Validation checks the prerequisites and configuration required for a controlled start. If validation succeeds, Servire reports that validation passed and enables the appropriate next action.

Lumen Service
Operational control plane for Lumen · [View Licence](#)

Service healthy

Stack Health

Stack State	Bootstrap	Managed Stack	External	Last startup
READY TO START	2 / 2 running	0 / 7 running	1 / 1 available	9.54 s

Stack Actions

Validate Start Stop Restart Clear Logs

Validation passed: 83 checks passed.
Last validation: 2026-09-23 20:55:08 UTC.

► Show validation details

Operational Log

Source	All sources	Severity	All	Stream	All	Search	Filter messages	500 shown · 500 buffered	Pause	Export	Clear
13:55:08	Lumen	SYSTEM	INFO								
13:55:08	Lumen	SYSTEM	INFO								
13:55:08	Lumen	SYSTEM	INFO								
13:55:08	Lumen	SYSTEM	INFO								
13:55:08	Lumen	SYSTEM	INFO								
13:47:17	Lumen	SYSTEM	INFO								
13:47:17	Lumen Vestigare	SYSTEM	INFO								
13:47:17	Lumen Vestigare	STDOUT	UNKNOWN								
13:47:17	Lumen Vestigare	SYSTEM	INFO								
13:47:17	Lumen Rogare	SYSTEM	INFO								
13:47:17	Lumen Rogare	STDOUT	UNKNOWN								
13:47:17	Lumen Rogare	SYSTEM	INFO								

Authorization

State	Last successful	License remaining	License expires	Host reserved
AUTHORISED	AUTHORIZED	23h 35m 27s	2026-09-24T20:30:41.985207Z	2026-09-24T02:30:42.309786Z

Figure 2. Servire after successful validation. The Operational Log shows the validation sequence and the Stack is ready to start.

Start

Select Start. Servire starts the managed components in dependency order and reports progress in the Operational Log. Do not treat an individual component becoming available as completion of the whole start sequence; wait for the Stack to reach its ready state.

Lumen Service
Operational control plane for Lumen · [View Licence](#)

Service healthy

Stack Health

Stack State	Bootstrap	Managed Stack	External	Last startup
READY	2 / 2 running	7 / 7 running	1 / 1 available	8.06 s

Stack Actions

Validate Start Stop Restart Clear Logs

Validation passed: 83 checks passed.
Last validation: 2026-09-23 20:55:41 UTC.

► Show validation details

Operational Log

Source	All sources	Severity	All	Stream	All	Search	Filter messages	500 shown · 500 buffered	Pause	Export	Clear
13:55:49	Lumen	SYSTEM	INFO								
13:55:49	Lumen	SYSTEM	INFO								
13:55:49	Lumen Praebere	SYSTEM	INFO								
13:55:49	Lumen Praebere	SYSTEM	INFO								
13:55:49	Lumen	SYSTEM	INFO								
13:55:49	Lumen	SYSTEM	INFO								
13:55:49	Lumen Rogare	STDER	INFO								
13:55:48	Lumen Rogare	STDER	INFO								
13:55:48	Lumen Rogare	STDER	INFO								
13:55:48	Lumen Rogare	STDER	INFO								
13:55:48	Lumen Rogare	STDER	INFO								
13:55:48	Lumen	SYSTEM	INFO								
13:55:48	Lumen Nuntius	STDOUT	INFO								

Authorization

State	Last successful	License remaining	License expires	Host reserved
AUTHORISED	AUTHORIZED	23h 34m 51s	2026-09-24T20:30:41.985207Z	2026-09-24T02:30:42.309786Z

Figure 3. Servire during/after Stack start-up with the Operational Log populated by component readiness events.

HEALTHY END STATE A normal start finishes with the managed components available and Lumen READY in the operational sequence.

4. Servire: health, dependencies and configuration

Managed Components and External Dependencies

The lower Servire workspace separates managed Lumen components from external dependencies. Managed component cards show implementation, dependencies, health, launch information, state and process information. External Dependencies shows the provider and persistence services that Lumen relies on.

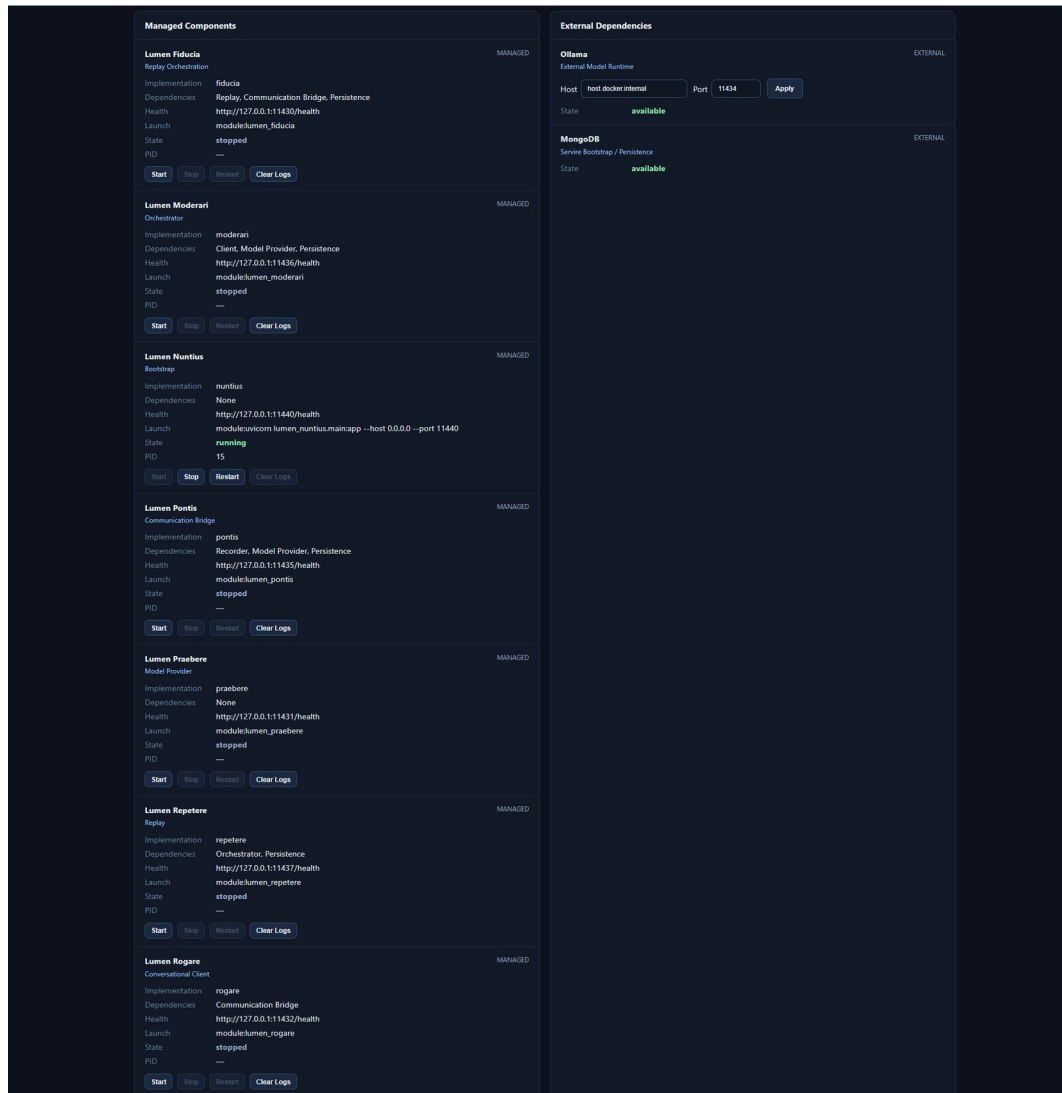


Figure 4. Managed Components alongside External Dependencies. Ollama and MongoDB are shown separately from the managed Stack.

Ollama

Set the Ollama host and port to the endpoint reachable from the Lumen container, select Apply, and confirm that the state becomes available. Provider discovery and model lifecycle are then handled through Praebere.

Configuration

The Configuration area provides the resolved runtime configuration for each component. Expand a component to inspect its implementation, working directory, launch command, configuration file, health endpoint, dependencies and interfaces. The detailed configuration control can be used where a component exposes it.

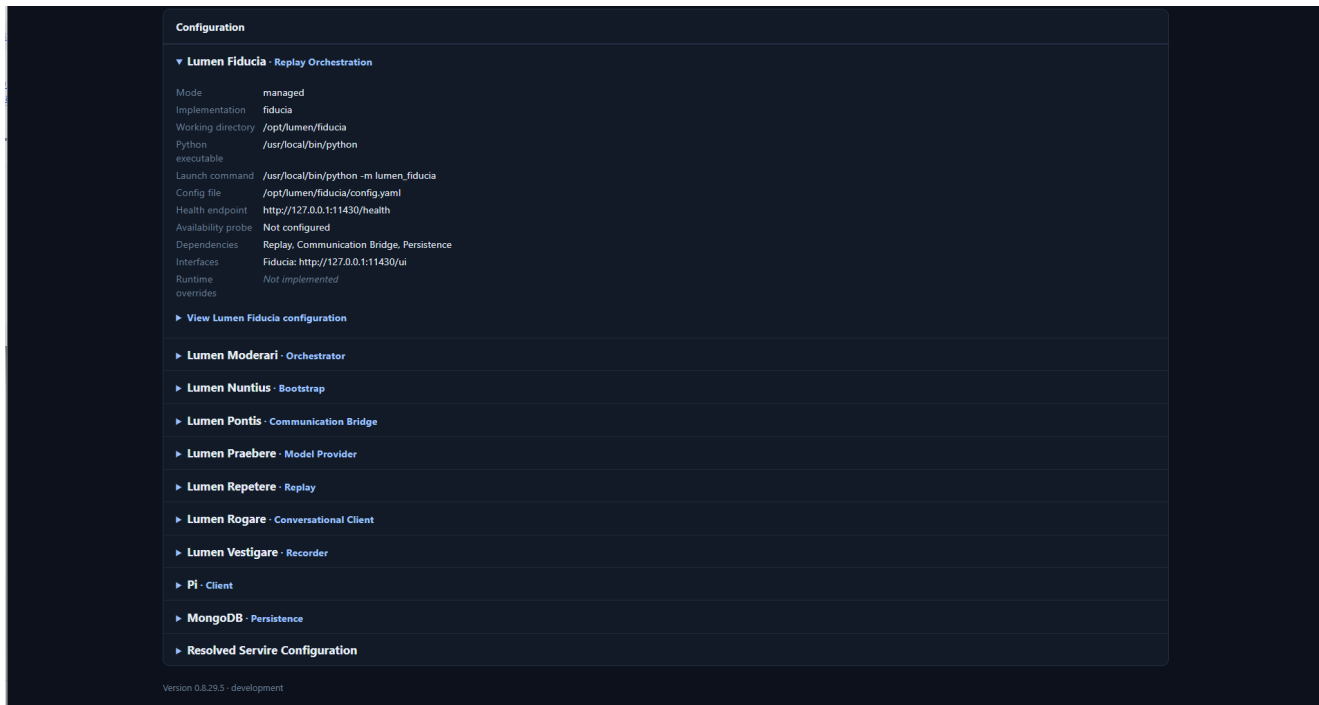


Figure 5. Configuration with Fiducia expanded as a representative component. The same inspection pattern applies to the other components.

USE CONFIGURATION FOR INSPECTION For normal research operation you should not need to edit internal component configuration. Use these views primarily to understand and diagnose the running M0.1 installation.

5. Nuntius: communication and bootstrap visibility

Nuntius provides visibility into Lumen service communication and bootstrap activity. Its interface is useful when confirming that requests are flowing between components and when investigating service-level communication.

The screenshot displays the Lumen Nuntius interface, which is part of the Lumen Servire workspace. The interface is divided into several sections:

- Service Bar:** A horizontal bar at the top showing various services: Servire, Fiducia, Moderari, Nuntius (selected), Pontis, Praebere, Repetere, Rogare, and Vestigare. Each service has a sub-label: Service, Replay Orchestration, Orchestrator, Bootstrap, Communication Bridge, Model Provider, Replay, Conversational Client, and Recorder respectively.
- Nuntius Status:** A green 'Available' button and an 'Open directly' link.
- Lumen Nuntius Control-plane routing diagnostics:**
 - HEALTH:** OK (green)
 - SERVIRE:** CONNECTED (green)
 - ACTIVE ROUTES:** 8
 - IN FLIGHT:** 0
 - CONTROL TIMEOUT:** 5s
- Recent Requests:** A table with columns: Started, Request, Origin / Session, Target, Result, and Elapsed. It shows a list of recent requests with their details.
- Request Detail:** A section on the right that allows selecting a request to inspect its lifecycle. It currently shows 'No selection'.
- Active Routes:** A table with columns: Service, Endpoint, and Listeners. It lists the active routes for each service.

Figure 6. Nuntius showing a connected state, recent requests, request detail and active routes.

What to look for

- A healthy/connected state at the top of the view.
- Recent Requests for request method, route, target and result.
- Request Detail when a request is selected.
- Active Routes to confirm the component routes known to Nuntius.

6. Praebere: model provider and model lifecycle

Praebere is the model-provider control surface. In M0.1 the provider is Ollama. Praebere discovers available models, tracks provider state and coordinates model selection/reservation for active Lumen work.

The screenshot shows the Lumen Servire Praebere interface. At the top, there's a navigation bar with tabs for various services: Servire, Fiducia, Moderari, Nuntius, Pontis, Praebere (selected), Repetere, Rogare, and Vestigare. Below this, the Praebere section is active, displaying a green 'Available' status and an 'Open directly' link. The main content area is titled 'Lumen Praebere' and includes a subtitle 'Authoritative provider and model runtime'. It features several key metrics: 'SELECTED MODEL' (None), 'SELECTION STATE' (available), 'RESERVATIONS' (0), and 'ACTIVE EXECUTIONS' (0). A 'Models' table lists discovered models with columns for MODEL, AVAILABLE, RESIDENT, PREFERRED, and SELECTED. The 'Provider' section shows details for 'ollama — running', including lifecycle, endpoint, preferred model, residency owner, and resident model. A 'Persistence and reconciliation' section displays reconciliation status, generation, and update timestamps. Finally, a 'Runtime actions' section provides buttons for 'Refresh Models', 'Reconcile with Pontis', 'Reset Runtime State', and 'Forced Recovery Reset'.

MODEL	AVAILABLE	RESIDENT	PREFERRED	SELECTED
devstral-24b	Yes	No	No	No
gemma3-4b	Yes	No	No	No
qwen2.5-coder:14b	Yes	Yes	No	No
qwen2.5-coder:14b-16k	Yes	No	No	No
qwen2.5-coder:14b-32k	Yes	No	Yes	No
qwen2.5-coder:14b-8k	Yes	No	No	No
qwen2.5-coder:7b	Yes	No	No	No
qwen3.635b-a3b	Yes	No	No	No

Figure 7. Praebere with Ollama available and the discovered model catalogue visible.

Model catalogue

The Models table shows the models discovered from the provider and their current availability/lifecycle information. Use Refresh Models when the Ollama catalogue has changed and you need Praebere to discover the current provider state.

Selected and reserved models

The screenshot displays the Lumen Praebere interface, which is part of the Lumen Servire ecosystem. The top navigation bar includes tabs for various components: Service, Fiducia, Moderari, Nuntius, Pontis, Praebere (selected), Repetere, Rogare, and Vestigare. Below the navigation bar, the Praebere section is active, showing a status of 'Available' and a button to 'Open directly'. The main content area is titled 'Lumen Praebere' and 'Authoritative provider and model runtime'. It features a 'SELECTED MODEL' dropdown set to 'qwen2.5-coder:7b', a 'SELECTION STATE' of 'locked', 'RESERVATIONS' of '1', and 'ACTIVE EXECUTIONS' of '1'. A green badge indicates 'Ollama available'. The interface is divided into several panels:

- Models:** A table listing various models with columns for MODEL, AVAILABLE, RESIDENT, PREFERRED, and SELECTED.

MODEL	AVAILABLE	RESIDENT	PREFERRED	SELECTED
devstral24b	Yes	No	No	No
gemma3:4b	Yes	No	No	No
qwen2.5-coder:14b	Yes	Yes	No	No
qwen2.5-coder:14b-16k	Yes	No	No	No
qwen2.5-coder:14b-32k	Yes	No	Yes	No
qwen2.5-coder:14b-8k	Yes	No	No	No
qwen2.5-coder:7b	Yes	Yes	No	Yes
qwen3.635b-a3b	Yes	No	No	No
- Provider:** Details about the Ollama provider, including lifecycle, endpoint, preferred model, residency owner, and resident model.
- Persistence and reconciliation:** Information about reconciliation status, generation, and update times.
- Runtime actions:** Buttons for 'Refresh Models', 'Reconcile with Pontis', 'Reset Runtime State', and 'Forced Recovery Reset'.
- Reservations and active executions:** A table showing active sessions.

SESSION	ORIGIN	MODEL	RESERVATION	EXECUTION
session-rogare-20260923-135914	unknown	qwen2.5-coder:7b	Reserved	Active

Figure 8. Praebere while a model is selected and reserved by active Lumen work.

A reservation indicates that the model is currently owned by an active execution/session lifecycle. Do not force model changes merely to clear a reservation; end the owning session or execution normally whenever possible.

PROVIDER RECOVERY If Ollama is stopped or becomes unreachable, Lumen may release affected lifecycle state. Restore Ollama, confirm provider health, and refresh the model catalogue before continuing.

7. Rogare: run a conversation

Rogare is the conversational client. It creates a Lumen session, selects a model through Praebere and sends conversation turns through the Lumen execution path.

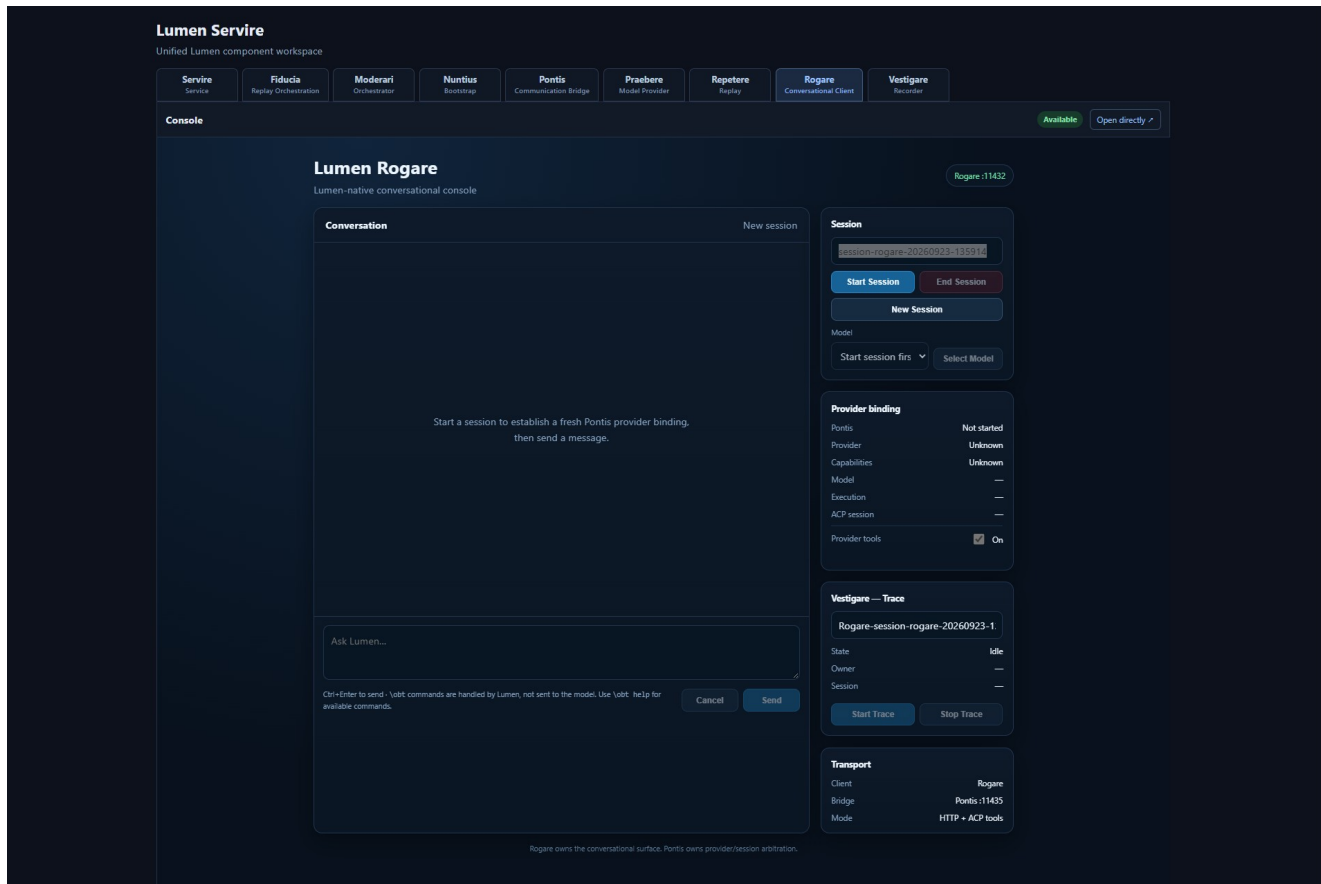


Figure 9. Rogare before a session is created.

Create a session and select a model

- 1 Select New Session.
- 2 Choose the required model from the model selector.
- 3 Confirm that Provider Binding and Bridge / Trace show the expected ready state before sending research prompts.

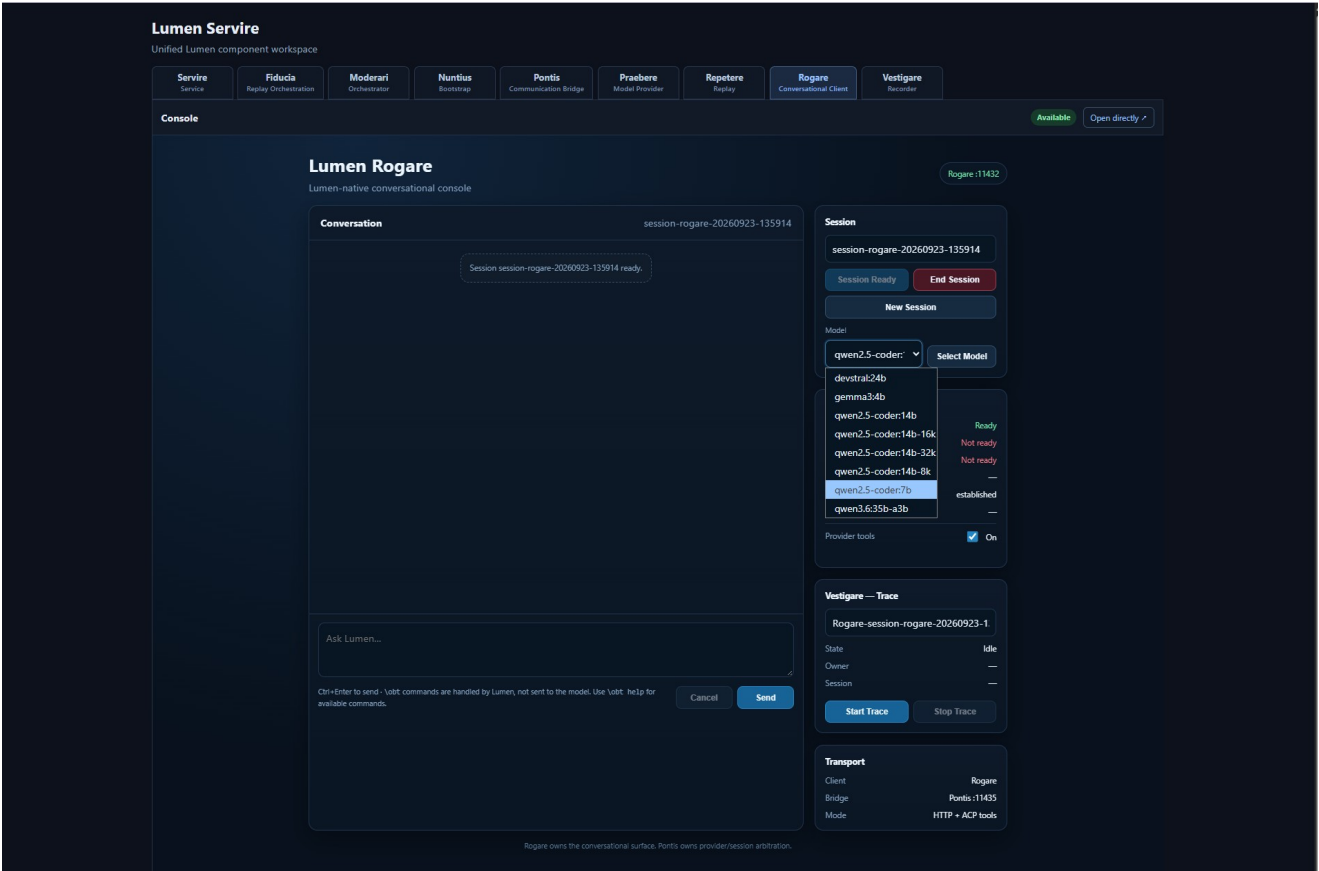


Figure 10. Rogare model selection with the available provider models shown.

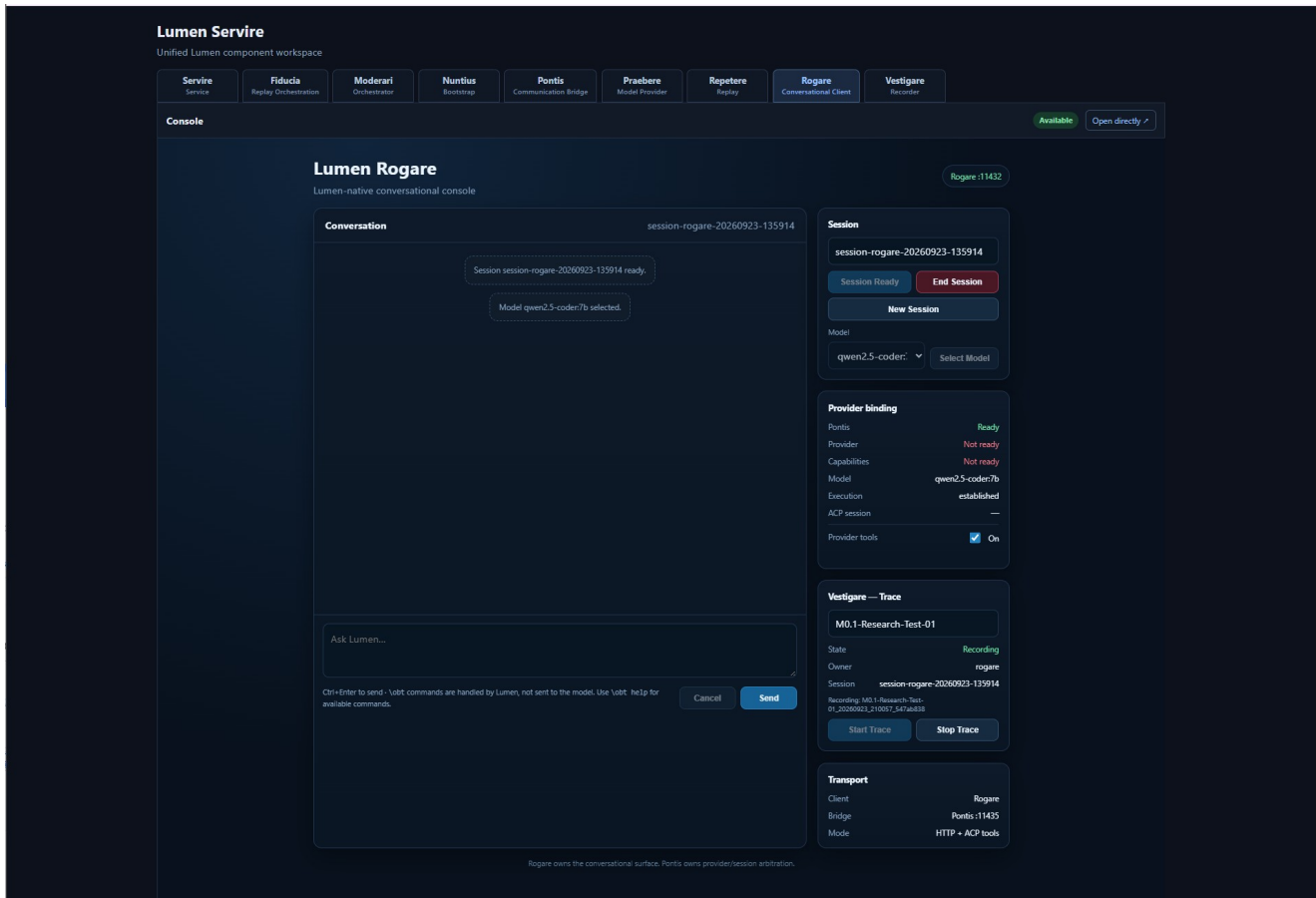


Figure 11. Rogare after model selection, with provider binding and bridge/trace state visible.

Send a prompt

Enter the prompt and select Send. The response appears in the conversation area. The right-hand panels retain the session, provider binding, bridge/trace and request context needed to understand the execution.

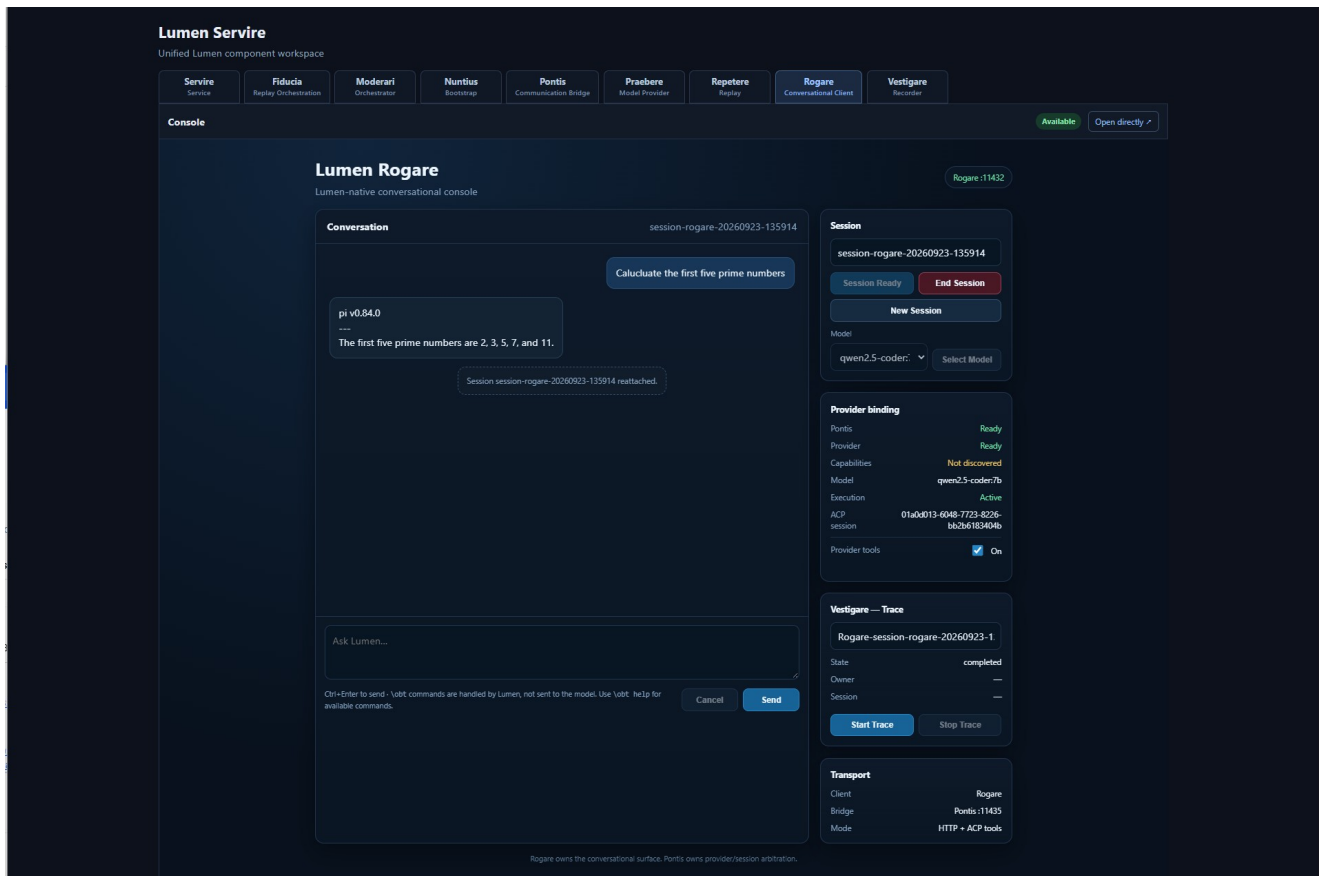


Figure 12. A completed Rogare interaction with the session and model state retained alongside the conversation.

End the session

Use End Session when the conversation is complete. Normal session termination allows Pontis and Praebere to release the execution and model lifecycle cleanly. Force-close controls are intended for exceptional recovery, not routine use.

8. Moderari and Pontis: inspect execution state

Moderari

Moderari exposes orchestration and system-prompt policy information for active sessions. The interface shows session state and the effective prompt policy applied to execution.

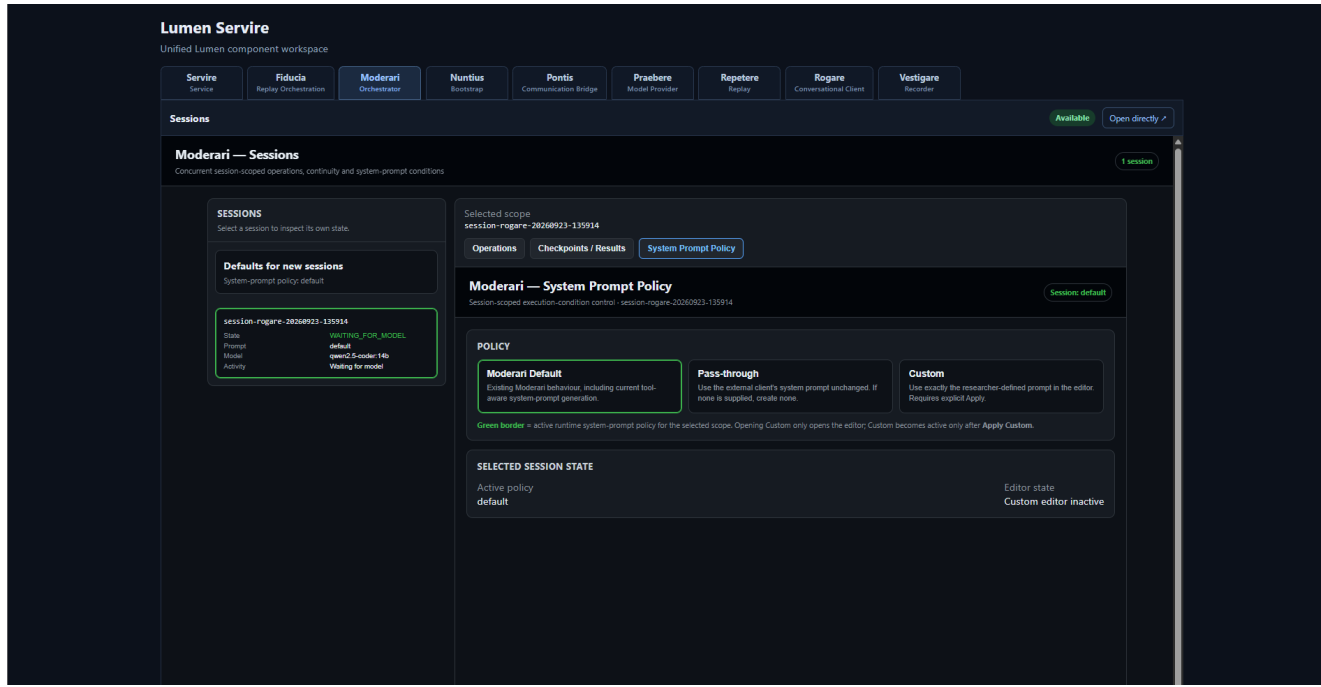


Figure 13. Moderari session view showing system-prompt policy and the active session context.

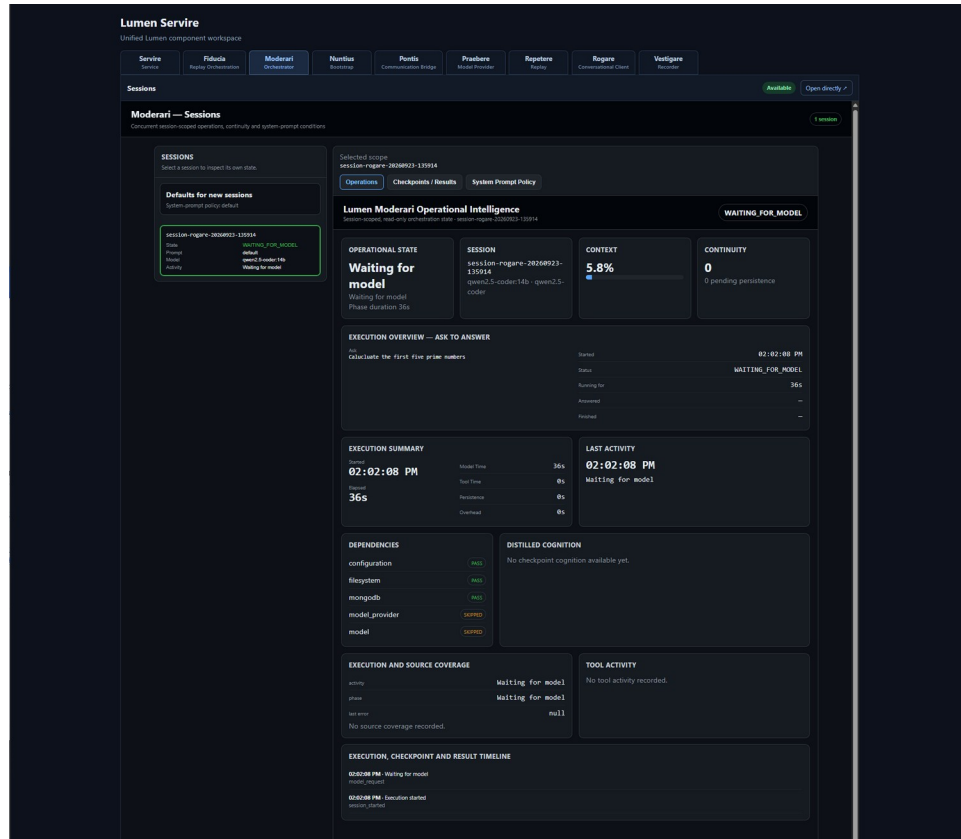


Figure 14. Moderari operational intelligence view for the active session.

Pontis

Pontis is the communication bridge between Lumen orchestration and Pi. Its UI is primarily an operational inspection surface: it shows active execution/session information and the current bridge state.

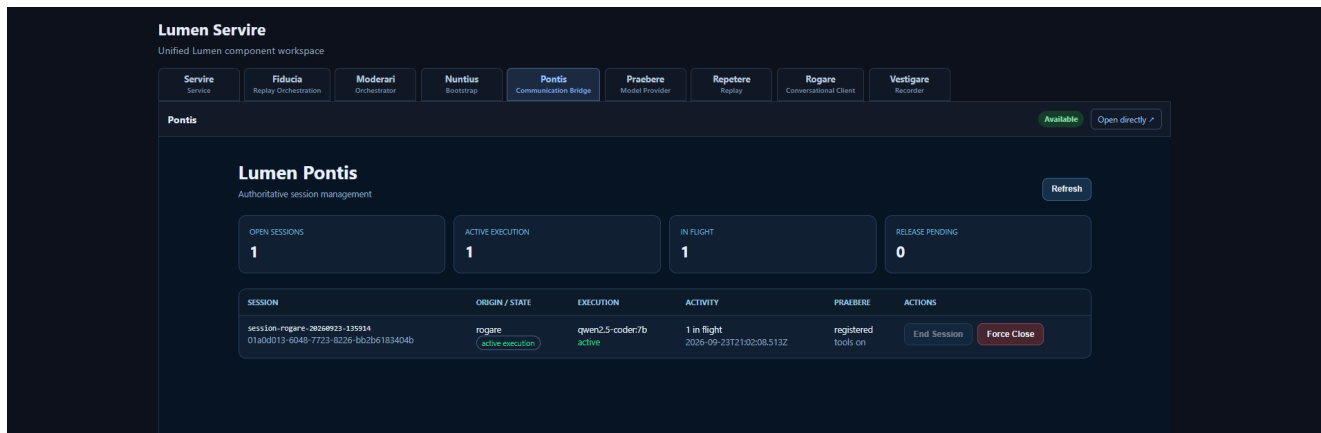


Figure 15. Pontis with an active session visible.

RESEARCHER GUIDANCE Rogare is the normal place to conduct a conversation. Moderari and Pontis are useful when you need to inspect how that conversation is being orchestrated and bridged.

9. Vestigare: record and inspect traces

Vestigare records the interaction path transparently. A completed recording becomes a trace that can be inspected directly and used as the source for Repetere replay experiments.

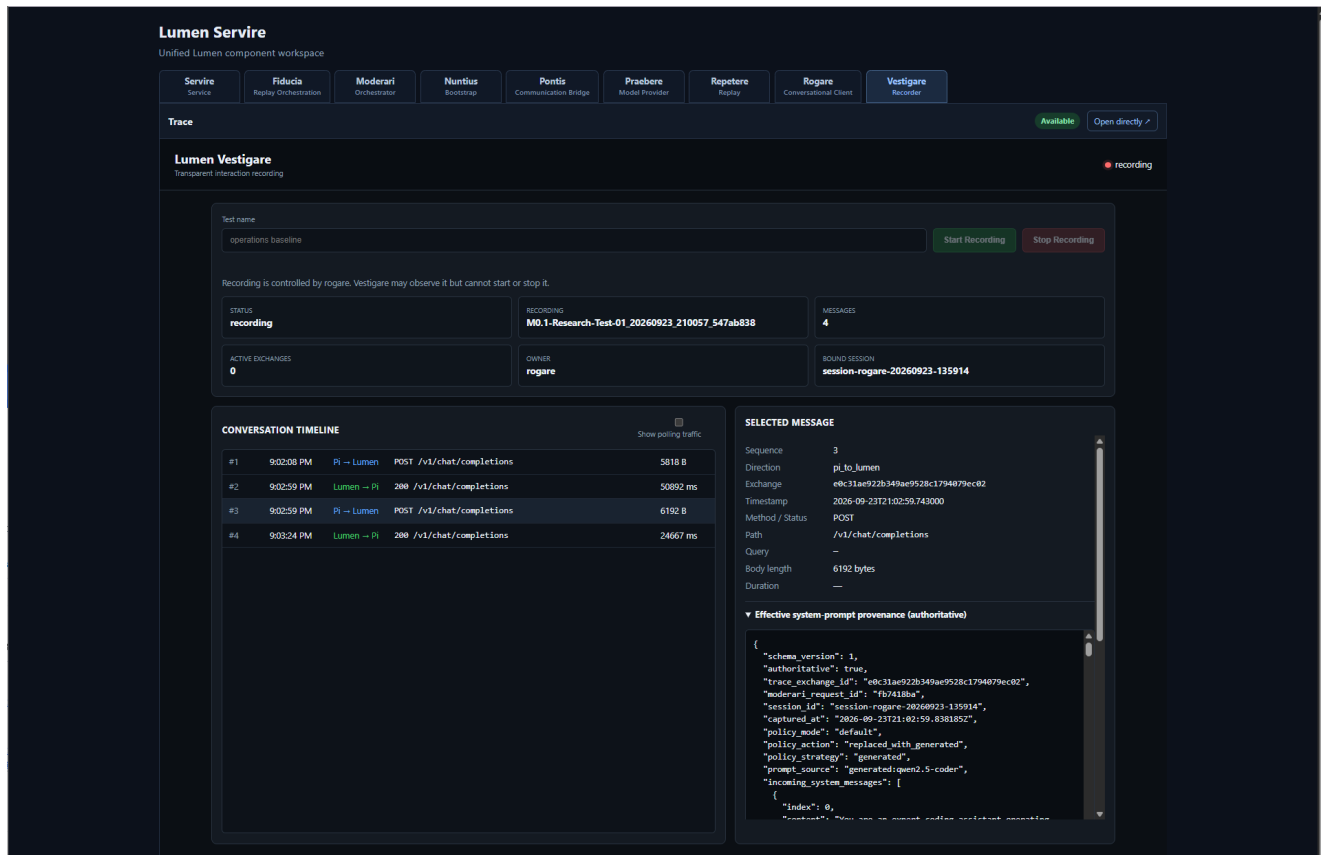


Figure 16. Vestigare with a completed trace, conversation timeline and selected-message detail.

Trace workflow

- 1 Give the recording a meaningful test name.
- 2 Start recording before the interaction you want to capture.
- 3 Run the target conversation or execution.
- 4 Stop the recording when the test is complete.
- 5 Inspect the Conversation Timeline. Select a message to inspect its recorded detail.

The trace records the ordered exchanges that passed through the observed path. The timeline makes direction, endpoint, timing and message-level detail visible without requiring the researcher to reconstruct the interaction from application logs.

10. Repetere: stage and replay a trace

Repetere turns a recorded trace into a controlled replay experiment. The Operator workflow is deliberately explicit: import a trace, stage it, run the replay, then review the result.

The screenshot displays the Lumen Repetere Operator interface. At the top, a navigation bar includes tabs for various components: Servire, Fiducia, Moderari, Nuntius, Pontis, Praebere, Repetere (active), Rogare, and Vestigare. Below this, a 'Replay' section shows 'MongoDB: Connected' and 'Target: Connected' status indicators. A summary dashboard shows 1 trace recording, 1 completed experiment, 0 staged experiments, and 4 total messages. The 'Operator workflow' section outlines four steps: 1. Inspect, 2. Stage, 3. Start, and 4. Review. The 'Trace recordings' table lists one recording: 'M0.1-Research-Test-01' with a status of 'Completed'. The 'Staged Replay experiments' section shows 'No replay experiments are staged.'

Lumen Repetere Operator

Stage a named experiment, run it, and inspect durable Run and child Trace evidence.

TRACE RECORDINGS 1 **COMPLETED** 1 **STAGED EXPERIMENTS** 0 **TOTAL MESSAGES** 4

Operator workflow Milestone 10

- 1. Inspect**
Review a completed Trace recording.
- 2. Stage**
Give the Experiment a unique name.
- 3. Start**
Replay while Repetere follows authoritative state.
- 4. Review**
Inspect Run history, fork evidence and child Traces.

Trace recordings 1 shown

RECORDING	MODEL	MESSAGES	EXCHANGES	DURATION	STATUS	ACTIONS
M0.1-Research-Test-01 M0.1-Research-Test-01_20250623_110557_547a6330-2026...	ollama/qwen2.5-coder:7b	4	2	206.61 s	Completed	Inspect Unique experiment name Stage Delete

Staged Replay experiments 0 shown

No replay experiments are staged.

► Technical configuration and capabilities

Figure 17. Repetere with a trace recording available and no experiment staged yet.

Stage the trace

Use the action on the trace row to stage the recording. The Staged Replay experiments area then exposes the experiment controls and replay model selection.

Lumen Servire
Unified Lumen component workspace

Service | Fiducia | Moderari | Nuntius | Pontis | Praebere | **Repetere** | Rogare | Vestigare

Replay | Available | Open directly ↗

LUMEN REPETERE

Lumen Repetere Operator

Stage a named experiment, run it, and inspect durable Run and child Trace evidence.

MongoDB: Connected
Connected to MongoDB database lumen

Target: Connected
Connected to moderari target at 'http://127.0.0.1:11438'

Experiment staged.

TRACE RECORDINGS
1

COMPLETED
1

STAGED EXPERIMENTS
1

TOTAL MESSAGES
4

Operator workflow

Milestone 10

1. Inspect
Review a completed Trace recording.

2. Stage
Give the Experiment a unique name.

3. Start
Replay while Repetere follows authoritative state.

4. Review
Inspect Run history, fork evidence and child Traces.

Trace recordings

1 shown

RECORDING	MODEL	MESSAGES	EXCHANGES	DURATION	STATUS	ACTIONS
M0.1-Research-Test-01 M0.1-Research-Test-01_20260923_210057_547ab838_2026...	ollama/qwen2.5-coder:7b	4	2	206.61 s	Staged	Inspect Already staged Stage Delete

Staged Replay experiments

1 shown

EXPERIMENT: M0.1-Research-Test-01 **Staged**
experiment-dcf8b3b57cd843e4a6006de0a034b447

Source Trace: M0.1-Research-Test-01_20260923_210057_547ab838

Run name: Repetere-M0.1-Replay

Source model: ollama/qwen2.5-coder:7b

Created: 2026-09-23 21:05:39 UTC

Replay model:
qwen2.5-coder:7b

Start | Unstage Experiment | Delete Experiment

Current / last run
No Runs yet.

► Technical configuration and capabilities

Figure 18. A trace staged as a replay experiment and ready to run.

Run the replay

Choose the replay model and start the run. Repetere reserves the required model through Praebere and displays current run state while the replay is executing.

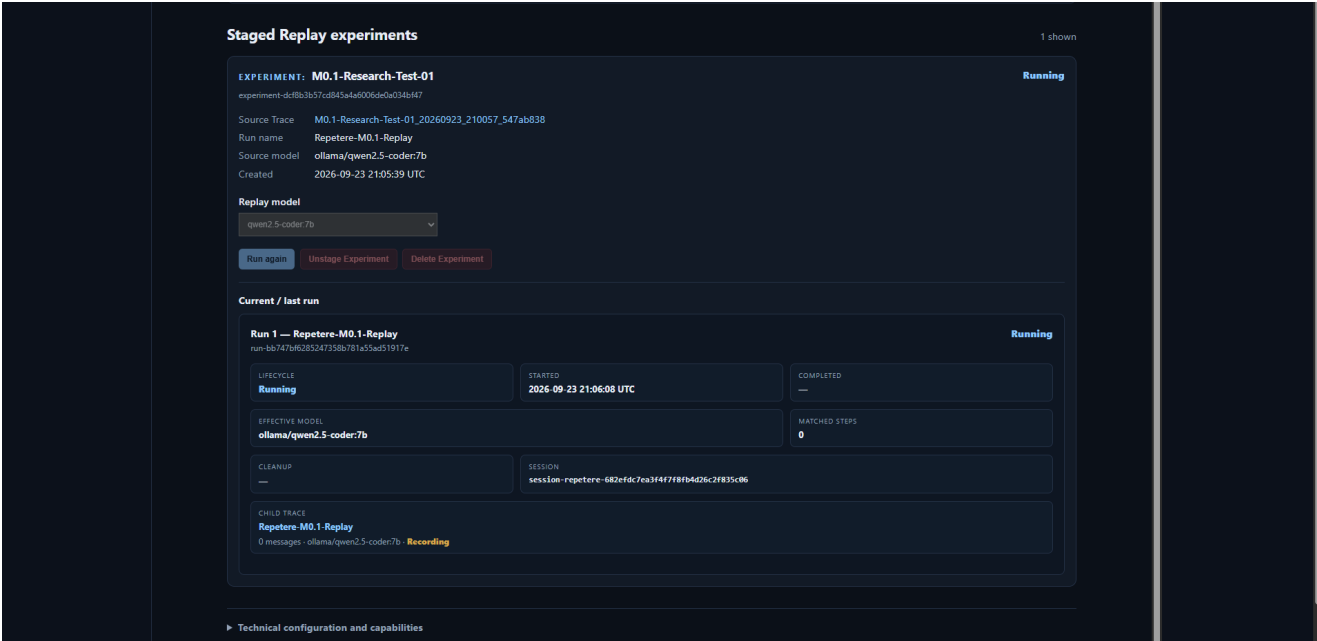


Figure 19. A staged experiment with an active replay run and current run state.

Review completion

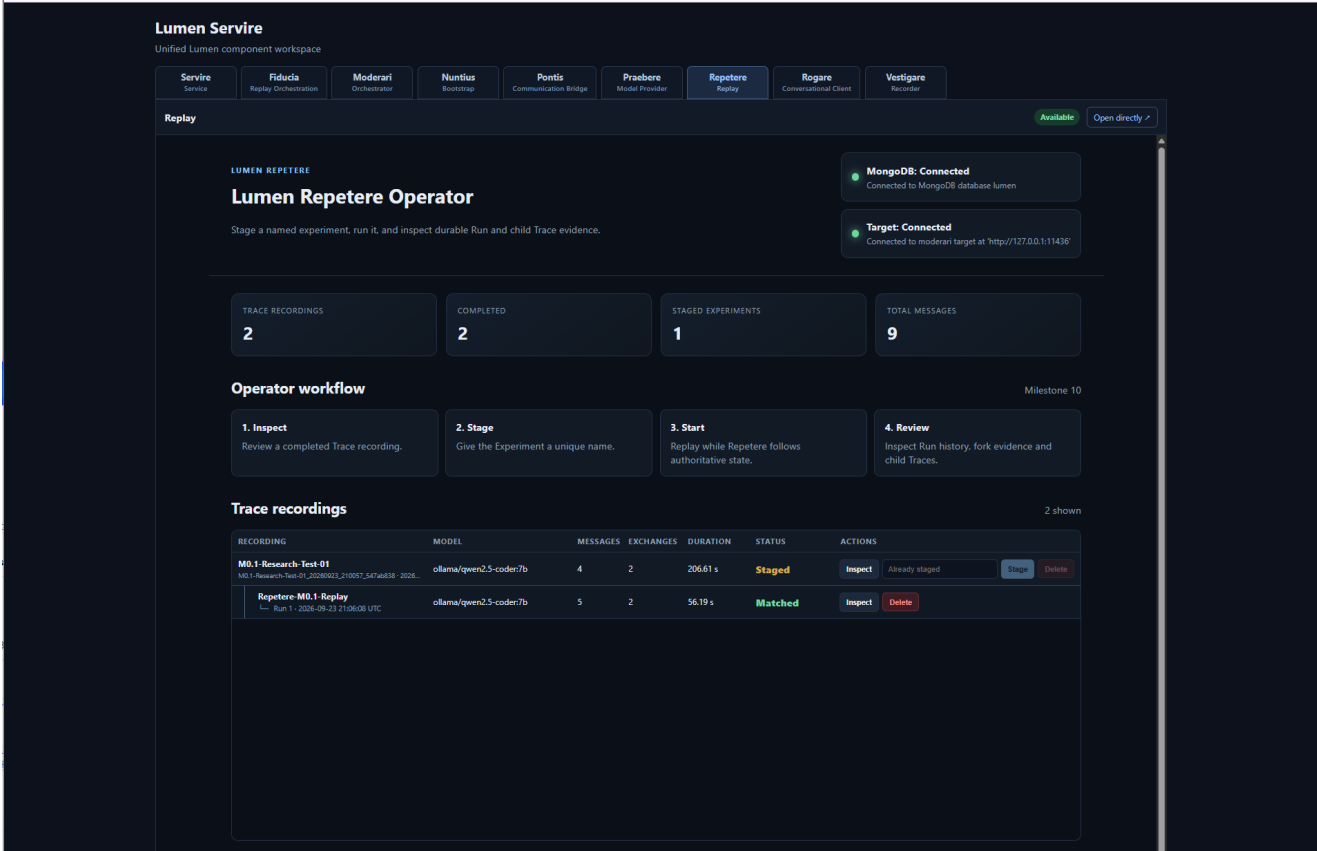


Figure 20. Repetere after replay completion, with the trace and experiment/run status visible.

A completed run preserves the experiment/run relationship so that the researcher can distinguish the original trace from subsequent replay executions. Where the replay diverges from the recording, the resulting child trace is retained as a separate trace relationship rather than overwriting the original evidence.

11. Repetere: model selection and repeat experiments

A staged experiment can be replayed against a different discovered model. This is useful for controlled comparisons while preserving the original trace as the common input.

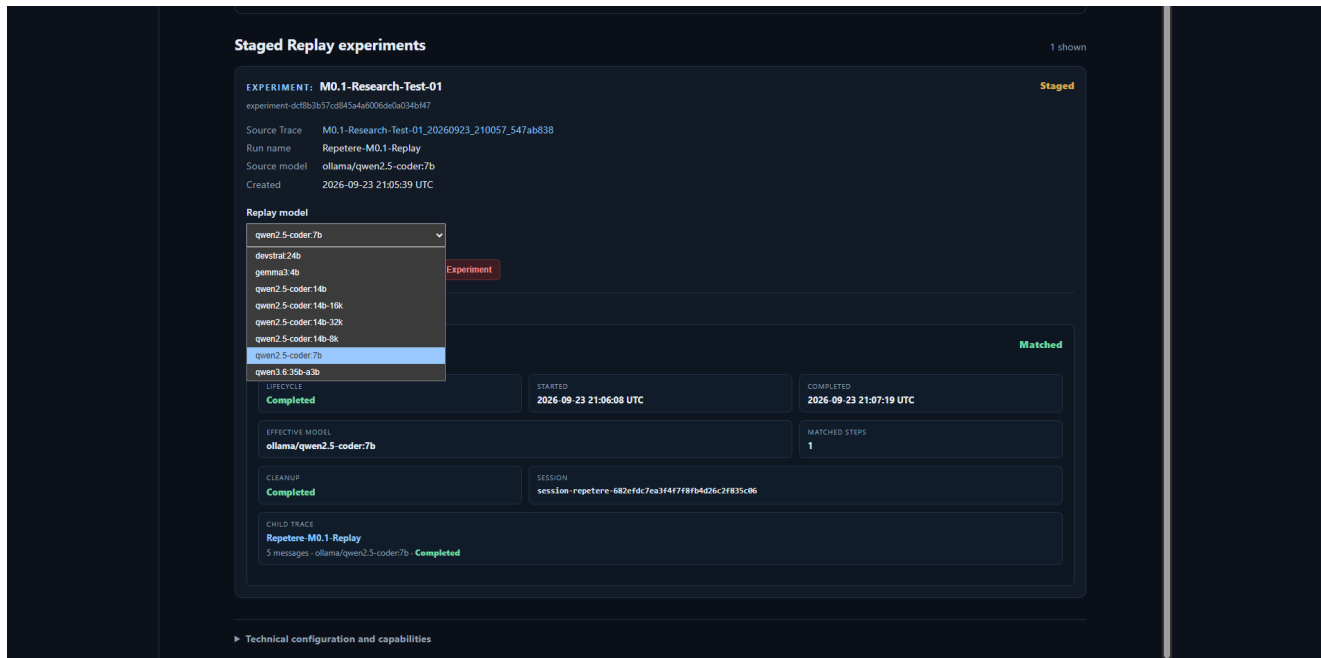


Figure 21. Replay model selector showing models available for a staged experiment.

Select the intended model before starting the next run. Repetere validates and reserves the model through the normal Lumen model lifecycle; the run should fail cleanly rather than silently substituting a different model if the requested model cannot be made available.

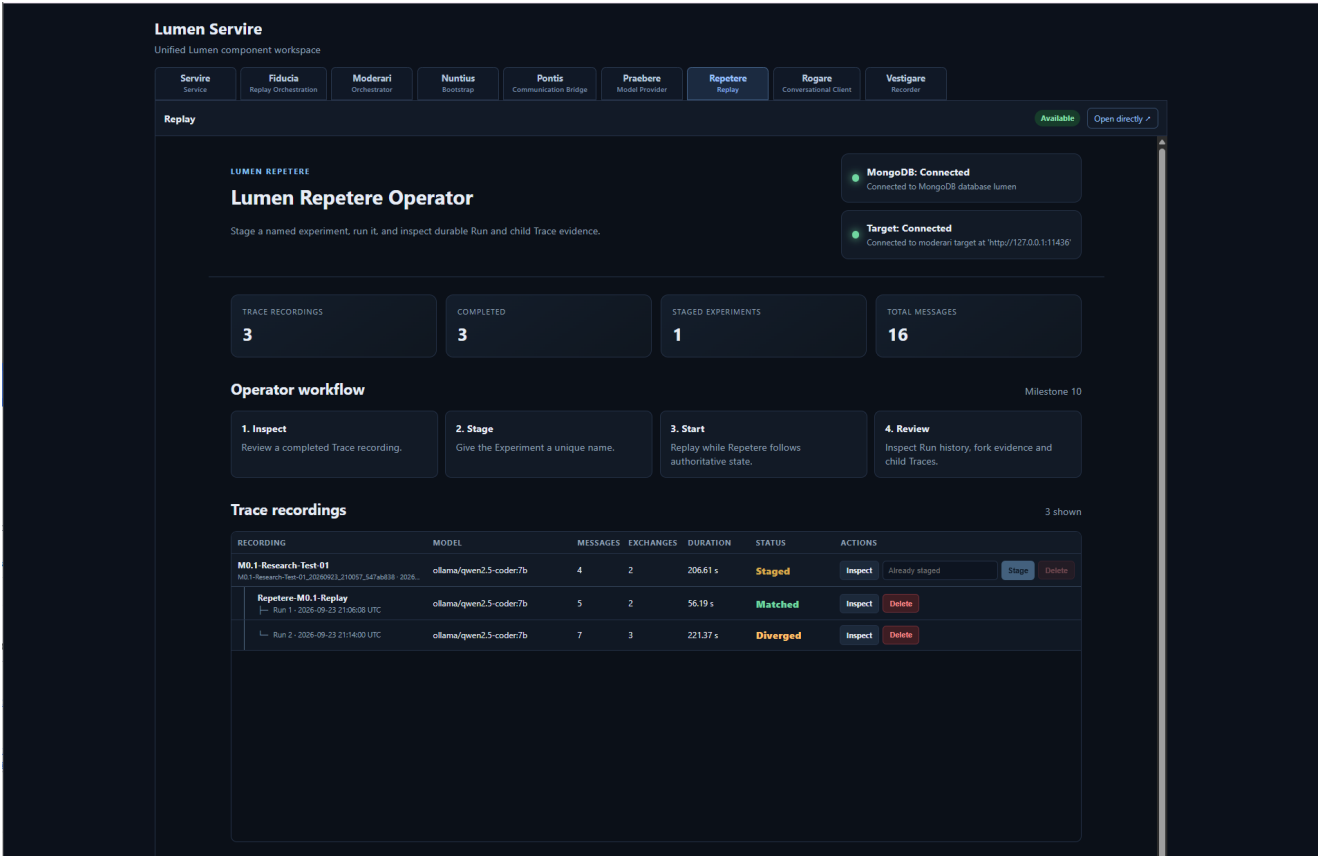


Figure 22. Repetere with multiple trace/replay records after repeated experimental work.

EXPERIMENTAL DISCIPLINE Use meaningful trace and experiment names and change one intended variable at a time. Lumen preserves provenance, but clear experimental naming makes later interpretation substantially easier.

12. Fiducia: schedule a prepared replay

Fiducia schedules prepared Repetere work. A replay must first be made available/prepared in Repetere; Fiducia then exposes it as a schedulable replay target.

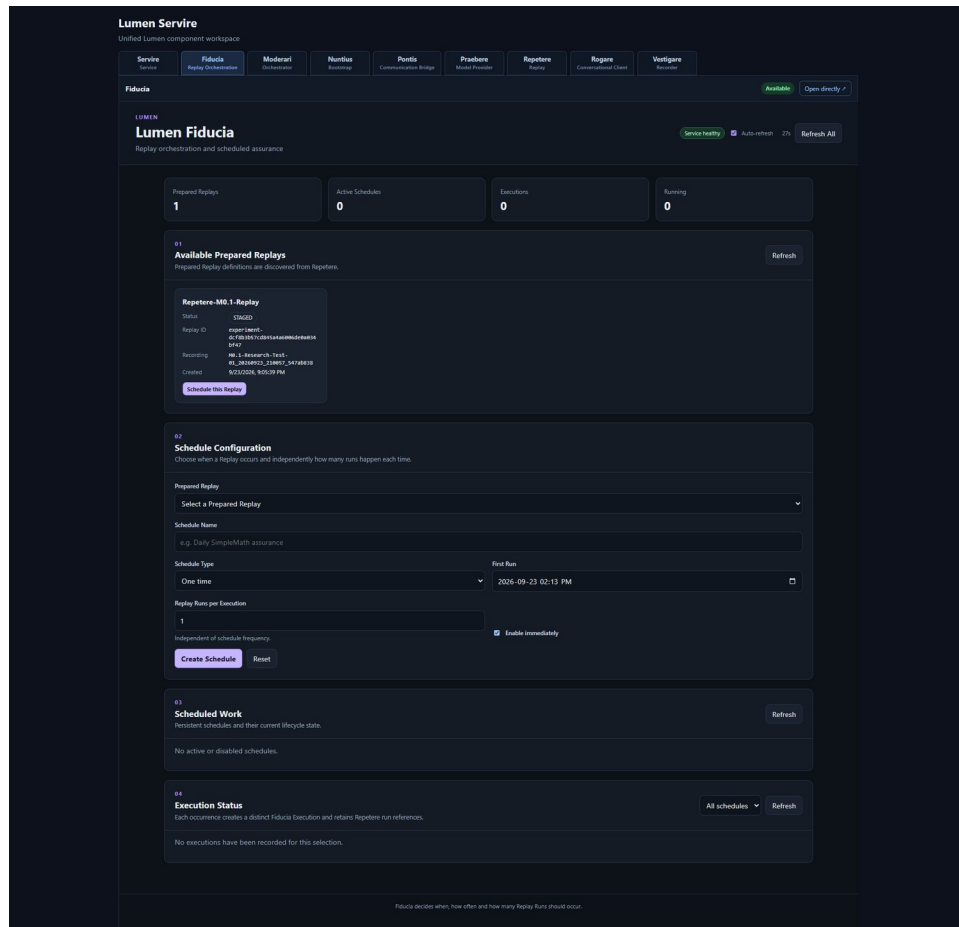


Figure 23. Fiducia with an available prepared replay and the Schedule Configuration form.

Create a schedule

- 1 Select the prepared replay.
- 2 Give the schedule a meaningful name.
- 3 Choose the schedule type and enter the required date/time or recurrence values.
- 4 Set the replay run limit as appropriate for the research task.
- 5 Select Create Schedule.

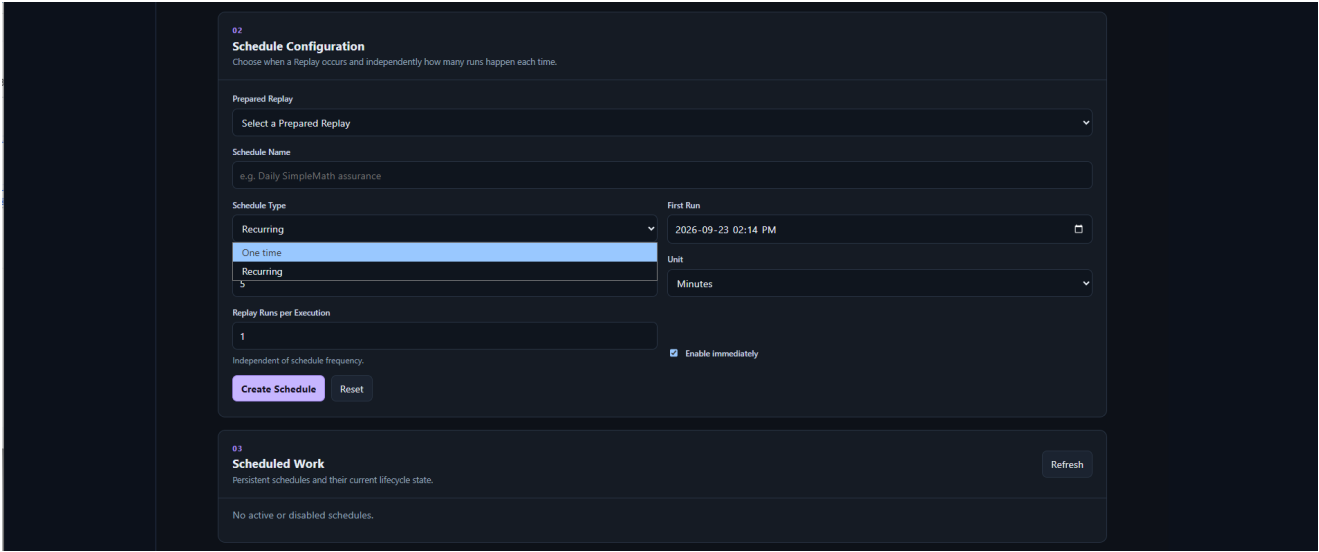


Figure 24. Fiducia schedule configuration with the schedule type selector open.

Scheduled work

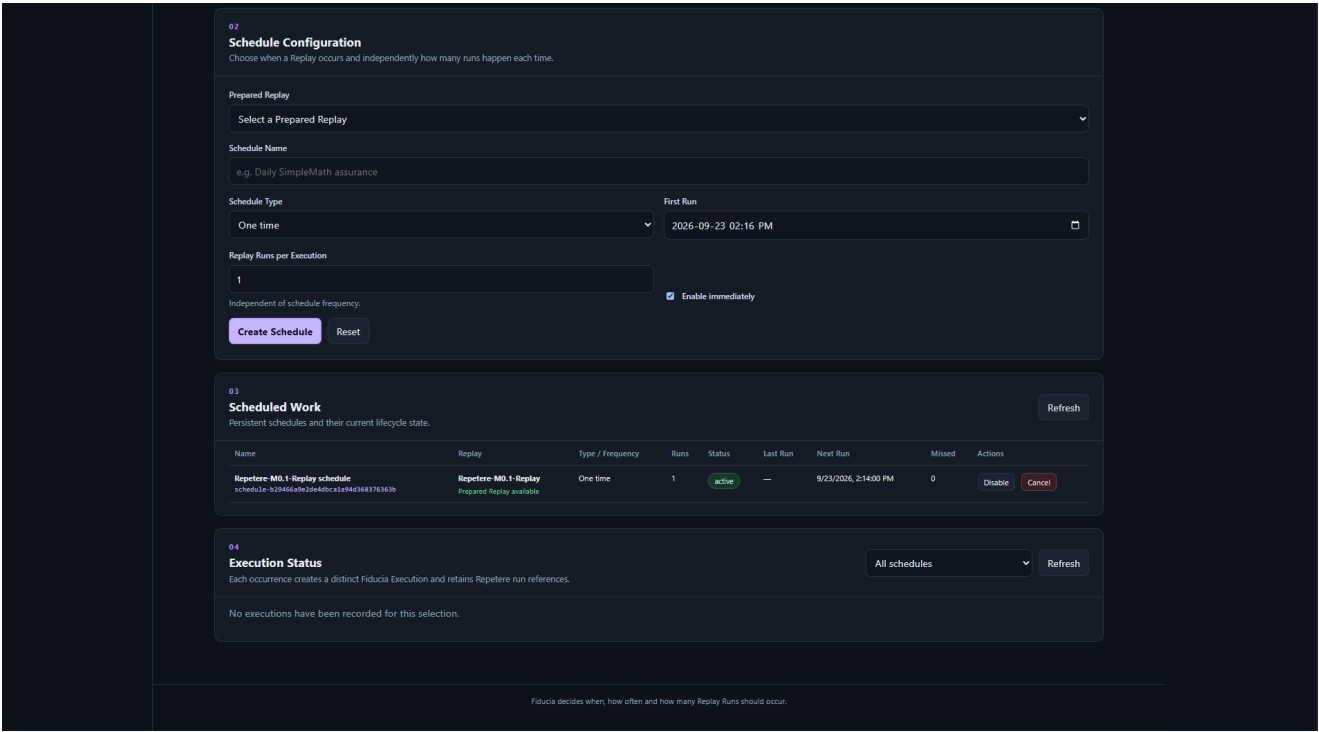


Figure 25. A newly created Fiducia schedule listed under Scheduled Work.

Scheduled Work shows the schedule definition and its next execution information. Use the controls in this area to manage the schedule rather than modifying the underlying replay experiment while a scheduled execution is pending.

13. Fiducia: execution status

When a schedule fires, Fiducia records the execution independently from the schedule definition. This makes it possible to see whether scheduled work ran, what it invoked and the resulting execution state.

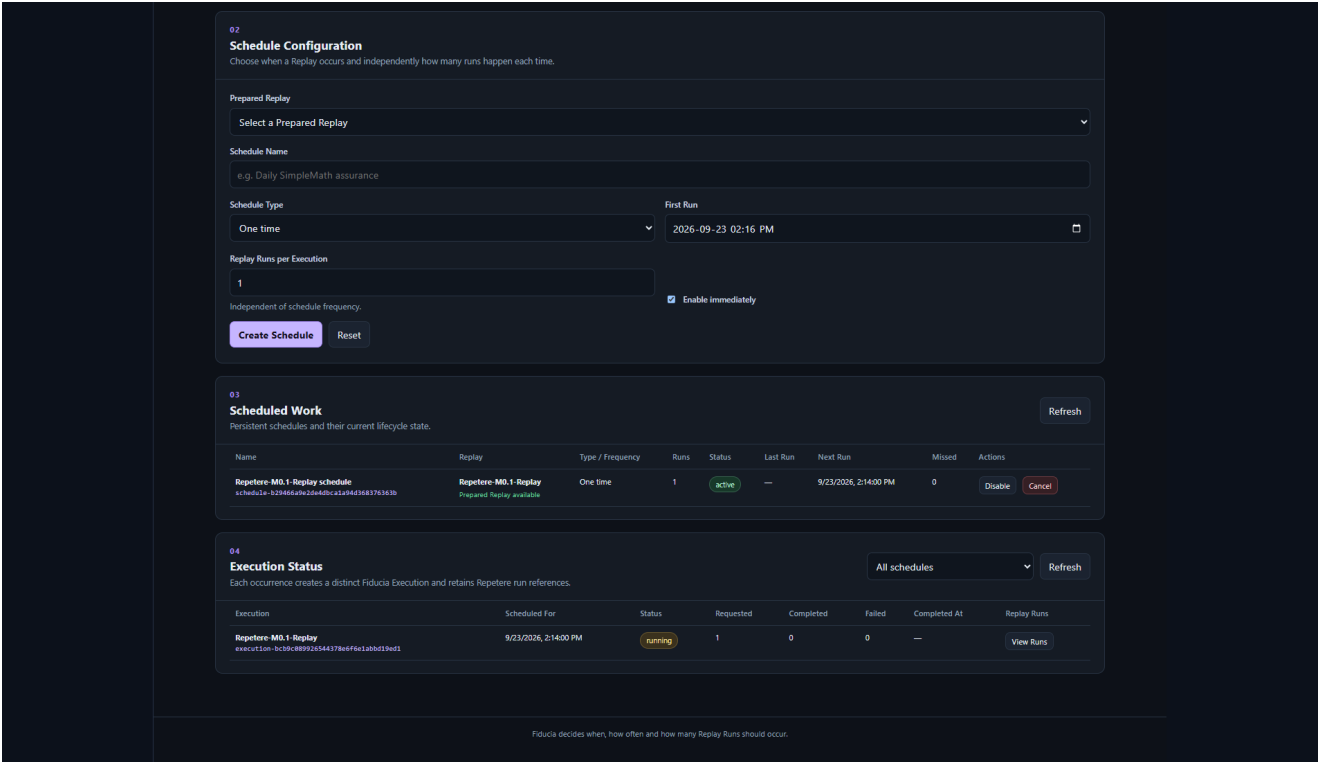


Figure 26. Fiducia showing scheduled work together with execution status.

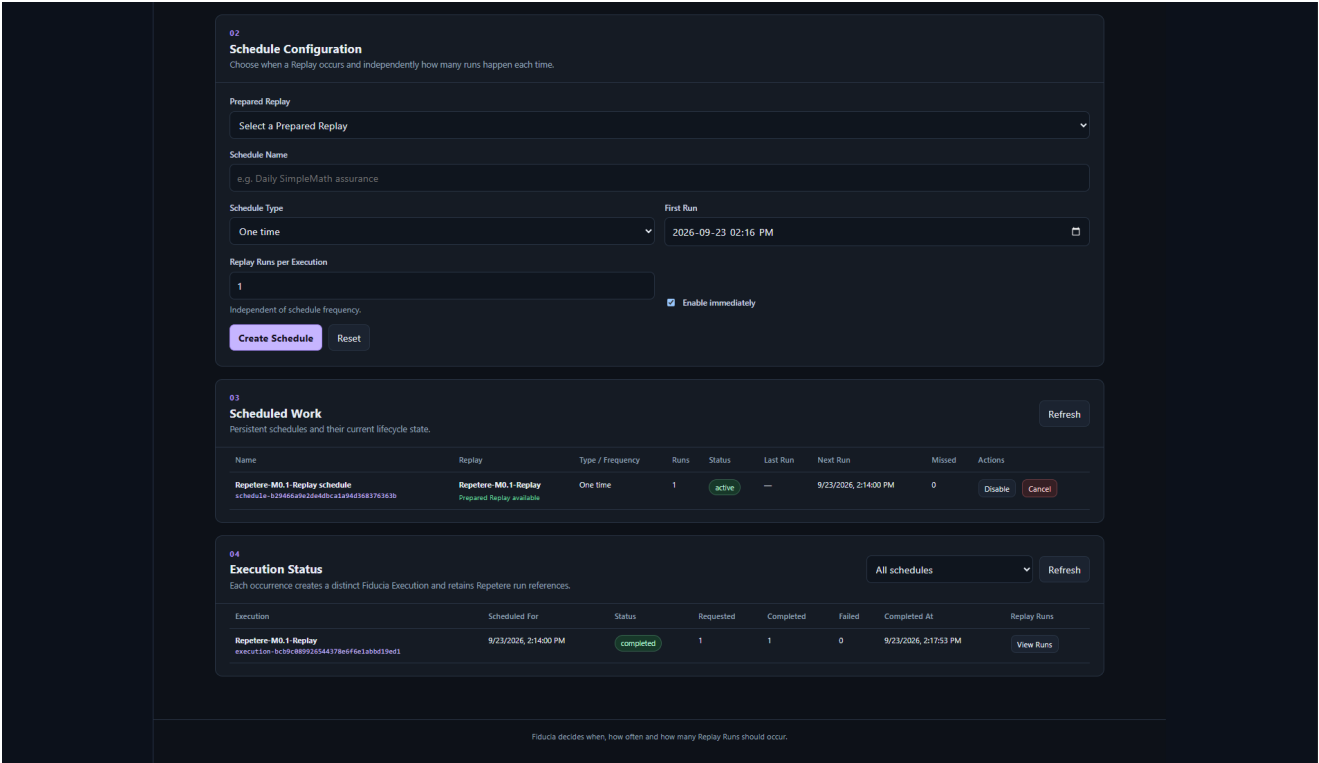


Figure 27. Fiducia after an additional scheduled execution, preserving both schedule and execution information.

Interpreting the view

- Scheduled Work describes what is planned.
- Execution Status describes what has actually run.
- Use the status and timing information together when investigating a missed or unexpected execution.
- Return to Repetere for replay-specific experiment/run detail.

14. Normal shutdown and restart

End active work first

Before stopping the Stack, end active Rogare sessions and allow replay or scheduled work to reach a terminal state where practical. This lets Pontis, Praebere and the orchestration components release their lifecycle state normally.

Stop the Stack from Servire

Use Servire lifecycle controls for normal managed Stack shutdown. The Operational Log shows component shutdown progress. If you are shutting down the complete Docker distribution, stop the managed Stack first and then stop the Compose application.

Stop Docker containers

From the distribution directory run: `docker compose down`

Do not add `-v` unless you intentionally want to delete the installation data volumes. A normal down/up cycle should preserve the local workspace, Servire data and MongoDB data.

Restart

Run `docker compose up -d`, open Servire, validate the external dependencies if required, and start the Stack. Periodic Lumen authorization is handled by the authorization mechanism; a temporary connectivity failure does not necessarily cause immediate loss of operation.

DO NOT DELETE VOLUMES CASUALLY Removing the Compose volumes is a destructive reset of the local installation state. Use it only when you deliberately want a clean installation test.

15. Troubleshooting

Servire opens but the Stack will not validate

- Confirm Ollama is running and reachable at the endpoint configured under External Dependencies.
- Confirm MongoDB reports available.
- Review the Operational Log for the first failed validation rather than only the final summary.
- If the provider catalogue changed, open Praebere and refresh the models after Ollama is healthy.

HAProxy container exits at startup

Confirm that haproxy.cfg exists as a normal file beside the Compose file. A missing bind-mount source can be created by Docker as a directory, which then causes a file-versus-directory mount failure when HAProxy starts.

Registry pull fails

- Run `docker login registry.illuminates.one` again using the researcher-specific registry credential.
- Confirm that the machine has network/DNS access to the registry hostname.
- If a VPN is active and DNS queries fail, test without the VPN or correct the VPN DNS path.

Authorization fails

Registry authentication and Lumen authorization are separate. A successful Docker pull does not authorize the installation. Confirm that the correct Lumen registration key was supplied and that the installation can reach Illuminates.One for authorization.

Known M0.1 limitation

DOCKER LOG EXPORT Operational log export is a known M0.1 limitation when Servire is running in the Docker distribution. Log viewing inside Servire remains available.

16. Operational reference

Normal researcher path

- 1 Start Docker Compose.
- 2 Open Servire and confirm authorization.
- 3 Confirm Ollama and MongoDB are available.
- 4 Validate the Stack.
- 5 Start the Stack and wait for Lumen READY.
- 6 Use Rogare for a conversation, optionally recording it with Vestigare.
- 7 Use Repetere to stage and replay a trace.
- 8 Use Fiducia when a prepared replay needs scheduled execution.
- 9 End active sessions/work, then stop the Stack when finished.

Research evidence chain

The researcher-facing flow can be read as an evidence chain: Rogare creates an interaction; Vestigare records it; Repetere replays it under controlled conditions; Fiducia can schedule prepared replay work; Servire provides operational control and health visibility across the complete system.

When to use each UI

Need	Open
Start, stop, validate, inspect logs	Servire
Inspect service communication	Nuntius
Inspect/select provider models	Praebere
Conduct a conversation	Rogare
Inspect orchestration policy/state	Moderari
Inspect bridge/session state	Pontis
Record or inspect a trace	Vestigare

Stage/run/review a replay	Repetere
Schedule a prepared replay	Fiducia

Appendix A. Release information

Product: Lumen M0.1 Research Distribution

Manual: SERVIRE USER MANUAL v0.1

Publisher: Illuminates.One

Release status: Research distribution / experimental

Manual date: September 2026

Distribution image

The M0.1 distribution is delivered from the private Illuminates.One registry. The researcher package uses Docker Compose to pull and run the frozen Lumen image together with the required supporting services.

Licence and authorization

Operation requires acceptance of the Lumen M0.1 Research Distribution Licence and periodic licence authorization with Illuminates.One. The Research Distribution must be able to communicate with Illuminates.One as required to validate its continuing authorization to operate.

Support information to retain

- Lumen/Servire version shown in the interface.
- Relevant Operational Log entries.
- Component and provider health state.
- Trace, experiment, run or schedule identifiers relevant to the issue.
- Whether the issue occurred before validation, during Stack start, during a session/replay, or during shutdown.